

Fundamentos de ciencia de datos con R - Módulo 7

Clase 3: k-Nearest Neighbors

CEPAL - Unidad de Estadísticas Sociales

2025-12-08

Introducción

El método de los **k vecinos más cercanos (k-Nearest Neighbors, k-NN)** es un algoritmo de aprendizaje supervisado muy sencillo e intuitivo.

La idea básica es:

*Para predecir el valor (o clase) de una observación nueva, miramos a sus **k vecinos más cercanos** en los datos y usamos lo que ocurre en ese vecindario.*

- ▶ Si la respuesta es **categorica** → votación de mayoría (clasificación).
- ▶ Si la respuesta es **numérica** → promedio de los vecinos (regresión).

Intuición del método k-NN



Idea principal

“Dime con quién andas y te diré quién eres”:

k-NN clasifica una observación según las características de sus vecinos más cercanos.

Intuición del método k-NN

Pasos generales:

1. Tomamos una observación nueva.
2. Medimos qué tan cerca está de cada persona del conjunto de entrenamiento.
3. Ordenamos por distancia.
4. Nos quedamos con los k más cercanos.
5. Para clasificación, hacemos un voto de mayoría.
6. Para regresión, sacamos un promedio.

Distancia y normalización

En k-NN, la noción de “cercanía” depende de cómo definimos la **distancia**.

La más usada es la **distancia Euclídea**:

$$d(i, j) = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \dots}$$

Problema:

- ▶ Si una variable está en una escala muy grande (por ejemplo, gasto en pesos) y otra en escala pequeña (edad en años), la variable grande domina la distancia.

Solución:

- ▶ **Estandarizar / normalizar** las variables numéricas antes de aplicar k-NN (por ejemplo, usando `scale()` en R).

Elección del parámetro k

El parámetro k controla la complejidad del modelo:

- ▶ k muy pequeño (ej. 1 ó 3):
 - ▶ El modelo es muy flexible, pero puede **sobreajustarse** (sensible al ruido).
- ▶ k muy grande (ej. 50, 100):
 - ▶ El modelo es más estable, pero puede **subajustarse** (pierde detalles).

En la práctica:

- ▶ Se prueban varios valores de k y se elige el que da mejor desempeño (ej. mejor exactitud en validación cruzada).

Historia: El Ministerio que quería anticipar el ingreso de los hogares

El Ministerio de Desarrollo Social llevaba años recopilando información de los hogares a través de encuestas. Sabían cosas como:

- ▶ si el hogar estaba en zona urbana o rural,
- ▶ el sexo y etnia de la persona jefa de hogar,
- ▶ la edad y años de escolaridad,

Sin embargo, había un problema:

Los formularios que estiman el ingreso real del hogar suelen llegar semanas o incluso meses después.

Historia: El Ministerio que quería anticipar el ingreso de los hogares

Esto generaba retrasos en:

- ▶ programas de transferencia monetaria,
- ▶ asignación de subsidios,
- ▶ monitoreo de brechas de ingreso,
- ▶ toma de decisiones rápidas en situaciones de emergencia.

Los analistas del Ministerio se preguntaron:



Pregunta central

¿Podemos predecir el ingreso de un hogar, usando únicamente sus características sociodemográficas, antes de recibir su reporte final?

Historia: El Ministerio que quería anticipar el ingreso de los hogares

La meta no era reemplazar el dato oficial, sino tener una estimación razonable para:

- ▶ planificar,
- ▶ simular escenarios,
- ▶ detectar hogares en riesgo.

Como querían un método simple, interpretable, sin supuestos fuertes, decidieron probar el algoritmo:

k-Nearest Neighbors (k-NN) “Hogares parecidos $\rightarrow$ ingresos parecidos”.

Historia: El Ministerio que quería anticipar el ingreso de los hogares

Usaremos la base `base_personas_gasto.rds` para predecir el **ingreso del hogar** como una variable numérica, a partir de sus características sociodemográficas.

```
head(datos, 10)
```

id_hogar	area	pobreza3	ingreso	sexo	etnia	edad_hh	anoest_hh
262	1	0	5391.527	Hombre	0	4	4
262	1	0	5391.527	Mujer	0	4	3
265	1	0	7077.083	Mujer	0	2	4
265	1	0	7105.557	Hombre	0	2	4
265	1	1	7077.083	Hombre	1	1	1
277	1	0	2418.750	Mujer	0	3	4
277	1	0	2390.277	Mujer	1	2	4
277	1	0	2390.277	Hombre	1	2	3
277	1	0	2418.750	Mujer	1	1	2
288	1	0	1375.000	Hombre	0	4	3

Separación en entrenamiento y prueba

Una vez que tenemos las covariables en formato numérico, dividimos la información en un conjunto de **entrenamiento** (para ajustar el modelo) y otro de **prueba** (para evaluar su desempeño).

```
set.seed(123)
n <- nrow(datos)
id_train <- sample(seq_len(n), size = 0.7 * n)

train <- datos[id_train, ]
test <- datos[-id_train, ]
```

Estandarización de las variables

Este método requiere que **todas las covariables sean numéricas**, porque k-NN se basa en calcular distancias entre observaciones.

Por esta razón, las variables categóricas como sexo y area deben ser transformadas a variables numéricas (por ejemplo, mediante dummies) antes de aplicar el algoritmo.

A continuación realizamos la transformación y la estandarización de las covariables.

Estandarización de las variables

```
# Matrices numéricas (model.matrix crea dummies para las categóricas)
x_train <- model.matrix(~ area + sexo + etnia + edad_hh + anoest_hh,
                        data = train)[, -1]
x_test  <- model.matrix(~ area + sexo + etnia + edad_hh + anoest_hh,
                        data = test)[, -1]

# Variable respuesta
y_train <- train$ingreso
y_test  <- test$ingreso
```

Nota

Las funciones `scale()` y `knn()` solo aceptan matrices numéricas.
Por eso primero usamos `model.matrix()` para transformar las variables categóricas (sexo, area) en dummies y luego estandarizamos.

Estandarización de las variables

```
x_train_s <- scale(x_train)
x_test_s  <- scale(
  x_test,
  center = attr(x_train_s, "scaled:center"),
  scale  = attr(x_train_s, "scaled:scale")
)
```

Zoom: ¿Quiénes son los 5 vecinos más cercanos?

Antes de usar k-NN automáticamente, haremos el cálculo manual paso a paso para entender exactamente qué hace el algoritmo:

```
i <- 59  
obs <- x_test_s[i, , drop=FALSE]  
test[i, ]
```

id_hogar	area	pobreza3	ingreso	sexo	etnia	edad_hh	anoest_hh
795	1	0	3524.115	Hombre	1	2	4

```
distancias <- apply(x_train_s, 1, function(fila) sqrt(sum((fila - obs)^2)))  
  
summary(distancias)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0	4.846603	5.578485	5.432806	6.045537	26.90519

Zoom: ¿Quiénes son los 5 vecinos más cercanos?

```
# Ordenamos
vecinos <- order(distancias)[1:5]

# Tabla de los 5 vecinos

tabla_vecinos <- train[vecinos,] %>% mutate(distancia = distancias[vecinos])
tabla_vecinos
```

id_hogar	area	pobreza3	ingreso	sexo	etnia	edad_hh	anoest_hh	distancia
48982	1	0	5276.5667	Hombre	1	2	4	0
54757	1	0	2941.5267	Hombre	1	2	4	0
38722	1	1	838.9233	Hombre	1	2	4	0
8664	1	0	2221.5267	Hombre	1	2	4	0
49617	1	0	3171.5267	Hombre	1	2	4	0

Zoom: ¿Quiénes son los 5 vecinos más cercanos?

```
# Predicción de ingreso a partir de los 5 vecinos (promedio)
ingreso_pred_i <- mean(tabla_vecinos$ingreso)

# Ingreso observado para esa observación del test
ingreso_obs_i <- y_test[i]

c(
  ingreso_observado = ingreso_obs_i,
  ingreso_predicho  = ingreso_pred_i
)
```

ingreso_observado	ingreso_predicho
3524.115	2890.014

Ajustar k-NN y evaluar

Ahora que vimos **paso a paso** cómo se construye el vecindario de 5 vecinos, usamos la función `knn.reg()` para obtener la predicción de ingreso para **todos** los hogares del conjunto de prueba.

```
library(FNN)
set.seed(123)
pred_knn <- knn.reg(
  train = x_train_s,
  test  = x_test_s,
  y     = y_train,
  k     = 5
)

ingreso_pred <- pred_knn$pred
```

Ajustar k-NN y evaluar

```
rmse <- function(obs, pred) sqrt(mean((obs - pred)^2))  
  
rmse_knn5 <- rmse(y_test, ingreso_pred)  
rmse_knn5
```

```
[1] 5541.089
```



Interpretación

El RMSE mide el error promedio de las predicciones de ingreso del modelo k-NN.
A menor RMSE, mejor capacidad predictiva.

k-NN para pobreza (clasificación)

```
# Variable respuesta (clasificación)
y_train_p <- factor(train$pobreza3)
y_test_p  <- factor(test$pobreza3)

library(class)
set.seed(123)

pred_knn_pob <- knn(
  train = x_train_s,
  test  = x_test_s,
  cl     = y_train_p,
  k      = 5
)
```

Evaluación del modelo (clasificación)

Matriz de confusión

```
# Variable respuesta (clasificación)
confusion_pob <- table(Observado = y_test_p, Predicho = pred_knn_pob)
confusion_pob
```

Observado/Predicho	0	1
0	4676	6
1	846	301

Exactitud

```
exactitud_pob <- mean(pred_knn_pob == y_test_p)
exactitud_pob
```

```
[1] 0.8538343
```

Evaluación del modelo (clasificación)



Interpretación

- ▶ La matriz de confusión muestra cuántos hogares pobres y no pobres fueron correctamente clasificados.
- ▶ La exactitud indica la proporción total de predicciones correctas del modelo k-NN.
- ▶ Como es una tarea de clasificación binaria, no usamos RMSE, sino métricas basadas en aciertos.

Ventajas y desventajas de k-NN

Ventajas	Desventajas
Muy simple de entender e implementar	Computacionalmente costoso con bases grandes
No asume una forma funcional del modelo	Depende fuertemente de la escala → requiere normalizar
Captura relaciones no lineales automáticamente	Sensible al ruido y a variables irrelevantes
Se adapta bien a formas complejas de decisión	Difícil de interpretar (no hay modelo explícito)
Útil para clasificación y regresión	Rendimiento disminuye en alta dimensionalidad (curse of dimensionality)

Resumen de la clase

Tema	Idea principal	Herramienta / Concepto
k vecinos más cercanos (k-NN) Distancia	Predice usando la información de los vecinos más similares Define la noción de “vecindad” entre observaciones	<code>class::knn()</code> , <code>FNN::knn.reg()</code> Distancia Euclídea
Normalización	Evita que variables grandes dominen la distancia	<code>scale()</code>
Elección de k	Controla el nivel de suavizamiento: k bajo sobreajusta, k alto subajusta	Prueba y error / validación
Evaluación en clasificación	Importa cuántos aciertos logra el modelo	Matriz de confusión, exactitud
Evaluación en regresión	Interesa el error promedio de la predicción	RMSE